

Name:

Roll:

Department:

CSL1010: Introduction to Computer Science

Major, IIT Jodhpur, Instructor: Palash Das, Date: 20.11.2025

Time: 180 Minutes

Full Marks: 50

Attempt all questions. Use the answer sheet for rough work and write your final answers neatly and clearly in the space provided on the question paper. Read each question carefully before attempting. **Good Luck!**

1. Fill in each blank with the correct output produced by the program.

5 * 2

a.

```
int Trial(int a, int b, int c)
{
    if ((a >= b) && (c < b))
        return b;
    else if (a >= b)
        return Trial(a, c, b);
    else
        return Trial(b, a, c);
}
```

Ans: The purpose of this function is to find _____ median (middle value _____ of three numbers.

b.

```
int main ()
{
    int i, j;
    char a[2][3] = {{ 'a', 'b', 'c' }, { 'd', 'e', 'f' }};
    char b[3][2];
    char *p = *b;
    for (i = 0; i < 2; i++){
        for (j = 0; j < 3; j++){
            *(p + 2*j + i) = a[i][j];
        }
    }
}
```

Ans: State the contents of the b array.

Enter your answer here:

a d
b e
c f

c.

```
char c[] = "LATE2005";
char *p = c;
printf("%s", p+p[3]-p[1]);
```

Ans: ___2005___ will be printed by the fragment of the code.

d.

```
int w (int n)
{
    static int i = 1;
    if (n >= 5) return n;
    n = n + i;
    i++;
    return w(n);
}
```

Ans: The value returned by w(1) function call will be _____7_____

e.

```
void main ()
{
    char a[6] = "hello";
    int i, j;
    for (i = 0, j = 5; i < j; a[i++] = a[j--]);
    printf ("%s\n", a);
}
```

Ans: The output will be _____empty_____

Name:

Roll:

Department:

2. Find the best matching for the following.

10 * 1

Column A	Column B
1. static char var;	a. Copies memory from one location to another.
2. m = malloc(10); m = NULL;	b. A variable located in the data section of memory.
3. char *ptr[10]	c. Causes programs to crash due to invalid memory access.
4. register int var1;	d. A lost memory which cannot be freed.
5. char *p; *p = 'a';	e. Prevents a variable's value from being modified after initialization.
6. const int	f. Using uninitialized pointers.
7. extern	g. Request to allocate a CPU register to store data.
8. Segmentation fault	h. Keyword to declare a variable defined in another file.
9. fseek()	i. Moves the file pointer to a specific location in a file stream.
10. memcpy()	J. Sequence of memory location to store addresses.

Ans: 1 → b, 2 → d, 3 → j, 4 → g, 5 → f, 6 → e, 7 → h, 8 → c, 9 → i, 10 → a

3. Study the pattern demonstrated in the example array and determine the sequence of operations required to produce the final output. Note that the example is only illustrative; the same procedure must work for arrays of any user-specified length. Based on your analysis, write a C program that implements this logic. [10]

2	5	1	4	3
---	---	---	---	---

Input

3	5	1	4	2
---	---	---	---	---

After Step 1

3	1	4	5	2
---	---	---	---	---

After Step 2

4	2	5	6	3
---	---	---	---	---

Final Output

Ans:

#include <stdio.h>

// Function to sort the middle elements (index 1 to n-2)

```
void sortMiddle(int arr[], int n) {
    for (int i = 1; i < n - 1; i++) {
        for (int j = i + 1; j < n - 1; j++) {
            if (arr[j] < arr[i]) {
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

Name:

Roll:

Department:

```
}
```

```
int main() {
    int n;

    // Read array size
    printf("Enter the number of elements: ");
    scanf("%d", &n);

    // Validate length
    if (n < 2) {
        printf("Array must have at least 2 elements.\n");
        return 0;
    }

    int arr[n];

    // Read array elements
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Swap first and last
    int temp = arr[0];
    arr[0] = arr[n - 1];
    arr[n - 1] = temp;

    // Step 2: Sort middle elements
    if (n > 2) {
        sortMiddle(arr, n);
    }

    // Step 3: Add 1 to each element
    for (int i = 0; i < n; i++) {
        arr[i] += 1;
    }

    // Output result
    printf("Final array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Name:

Roll:

Department:

4. A stack is a linear data structure (like an array) where both insertion and deletion of an element take place at the same end, known as the **top**. New elements are added (pushed) at the top, and existing elements can only be removed (popped) from the top. This follows the **LIFO (Last In, First Out)** rule, where the most recently added item is removed first. You can easily implement this behavior using a 1-D array by using a variable like top to track the index of the last inserted element. A code that implements is partially given below. You complete the code by writing three functions: **push()**, **pop()**, and **display()** to show the elements of the stack. [15]

```
#include <stdio.h>
#define MAX 100 // Maximum size of the stack

int stack[MAX];
int top = -1;

int main() {
    int choice, value;

    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n2. Pop\n3. Display\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Exiting program.\n");
                return 0;

            default:
                printf("Invalid choice! Please try again.\n");
        }
    }
}
```

```
void push(int value) {
    if (top == SIZE - 1) {
        printf("Stack Overflow\n");
    } else {
        top++;
        stack[top] = value;
        printf("%d pushed into stack\n", value);
    }
}

void pop() {
    if (top == -1) {
        printf("Stack Underflow\n");
    } else {
        printf("%d popped from stack\n", stack[top]);
        top--;
    }
}
```

Name:

Roll:

Department:

```
void display() {  
    if (top == -1) {  
        printf("Stack is empty\n");  
    } else {  
        printf("Stack elements: ");  
        for (int i = top; i >= 0; i--) {  
            printf("%d ", stack[i]);  
        }  
        printf("\n");  
    }  
}
```

5. Write True or False for each of the following statements.

5 * 1

- a. In C, `const int *p` makes the pointer `p` constant.
- b. If a pointer is incremented (e.g., `ptr++`), the pointer always increases by 1 byte.
- c. Accessing an array element out of bounds is guaranteed to cause a runtime error.
- d. In C, the statement `int x = (int)3.9 + (int)4.1;` will always give 7 because type casting truncates the decimal part.
- e. Using `malloc` to allocate memory and then freeing it twice (`free(ptr); free(ptr);`) is safe if `ptr` is non-NULL.

Ans: a. False, b. False, c. False, d. True, e. False